

THE COMPLETE HANDS-ON GUIDE

Ollama

Run Large Language Models on Your Own Machine

A practical, project-driven path to mastering Ollama — from your first install to production deployment. Every concept is explained from the ground up and grounded in the official Ollama documentation, so you never have to look elsewhere.

PHASE 1 OF 6

Setup & Installation

Phase 1 — Setup & Installation

This is where everything begins. By the end of this phase you will have a working local AI engine on your computer, you will understand exactly what is running behind the scenes, and you will be able to download, run, inspect, and delete models with a handful of simple commands — all without sending a single byte to the cloud.

What is Ollama, in one sentence? Ollama is a free, open-source tool that lets you download and run powerful open large language models (LLMs) — such as Llama, Gemma, Qwen, Mistral and DeepSeek — directly on your own hardware, with a clean command line, a built-in local server, and official Python and JavaScript libraries.

Because the models run on your machine, your prompts and data never leave your computer (unless you explicitly choose a cloud model). That means **privacy, zero per-token cost, and full offline capability** — the three reasons most people switch to a local setup.

WHAT YOU'LL LEARN IN THIS PHASE

Installing Ollama on Linux, macOS and Windows · understanding the background service on port 11434 · pulling, running, listing and removing models · checking what's loaded in memory · where models live on disk · and the different categories of models you can run.

1.1 Installing Ollama (Linux / macOS / Windows / Docker)

1.2 The Ollama service — a local server on port 11434

1.3 Pulling models — `ollama pull`

1.4 Running models in the terminal — `ollama run`

1.5 Listing installed models — `ollama list`

1.6 Removing a model — `ollama rm`

1.7 Checking running models — `ollama ps`

1.8 Where models are stored on disk

1.9 The different types of Ollama models

1.10 Hands-on project: your local AI lab

1.1 — Installing Ollama

Ollama ships as a small native application for all three major operating systems. The installer sets up two things: the **command-line tool** (`ollama`) that you type commands into, and a **background service** that does the actual model loading and text generation. You install once, and Ollama quietly runs in the background from then on.

macOS

The simplest route is the one-line install script, which detects your system and sets everything up. Open the **Terminal** app and paste:

```
# One-line install (recommended)
curl -fsSL https://ollama.com/install.sh | sh
```

Alternatively, download the `Ollama.dmg` from ollama.com/download, open it, and drag the Ollama app into your **Applications** folder. On macOS, Ollama appears as a small icon in the menu bar once running.

Windows

Open **PowerShell** and run the official install command, or download the `OllamaSetup.exe` installer and double-click it:

```
# Paste into PowerShell
irm https://ollama.com/install.ps1 | iex
```

After installation, Ollama runs automatically and appears in the Windows system tray. You can then use it from **Command Prompt**, **PowerShell**, or Windows Terminal.

Linux

Linux uses the same one-line script as macOS. The script auto-detects your distribution and, where possible, configures GPU acceleration:

```
# Works on most distributions (Ubuntu, Debian, Fedora, Arch, etc.)
curl -fsSL https://ollama.com/install.sh | sh
```

GPU ACCELERATION ON LINUX

For NVIDIA GPUs, install the official NVIDIA drivers and the NVIDIA Container Toolkit so Ollama can use your GPU. Ollama automatically detects compatible hardware — if it finds a supported GPU it uses it; otherwise it falls back to your CPU and system RAM. CPU-only works fine for smaller models, just more slowly.

Docker

If you prefer containers, the official image `ollama/ollama` is on Docker Hub:

```
# Run Ollama in a container, exposing the API on port 11434
docker run -d -v ollama:/root/.ollama -p 11434:11434 --name ollama ollama/ollama
```

Verify the installation

Whatever method you used, confirm it worked by opening a terminal and typing:

```
ollama
```

If installed correctly, you'll see a help screen listing available commands. You can also check the version explicitly:

```
ollama --version
ollama version is 0.23.0
```

KEEPING OLLAMA UP TO DATE

On macOS and Windows, Ollama downloads updates automatically — click the menu-bar / tray icon and choose "Restart to update". On Linux, simply re-run the install script (`curl -fsSL https://ollama.com/install.sh | sh`) to upgrade.

1.2 — Understanding the Ollama service (the local server on port 11434)

This is the single most important concept in the whole guide, so it's worth slowing down. Ollama is built as a **client-server** system, and understanding the split makes everything else click into place.

Component	What it does
The server (<code>ollama serve</code>)	A background program that loads models into memory and generates text. It exposes a local web API (REST) that anything on your machine can talk to.
The client (the <code>ollama</code> command, the Python library, your apps)	Sends requests to the server — "pull this model", "answer this prompt" — and displays the results. The client itself never loads a model; it just talks to the server.

When you installed Ollama, the server started automatically. It listens on **127.0.0.1 (localhost) port 11434** by default. Every command you run and every Python script you write ultimately sends an HTTP request to this address.

WHY 11434?

It spells "LLAMA" on a phone keypad (L=5... close enough — it's the project's playful default). You'll see `localhost:11434` everywhere in Ollama, so it's worth memorising.

You can prove the server is running by visiting it directly. Open a terminal and ask it for a heartbeat:

```
# Ask the running server if it's alive
curl http://localhost:11434
Ollama is running
```

If you ever need to start the server manually (for example after stopping it, or on a headless Linux box), use:

```
ollama serve
```

To see every environment variable that tweaks the server's behaviour, append `--help`:

```
ollama serve --help
```

KEY IDEA TO CARRY FORWARD

In Phase 2 you'll call Ollama from Python, and in Phase 6 you'll call the raw REST API. Both do exactly what the `ollama` command does under the hood: send an HTTP request to `localhost:11434`. There is no magic — just one local server and many ways to talk to it.

1.3 — Pulling models: `ollama pull`

Before you can run a model, you have to download it. The `pull` command fetches a model from the official model library at ollama.com/library and stores it locally. Think of it like `git pull` or `docker pull` — it grabs the files once, then they live on your disk.

```
# Download Meta's Llama 3 model
ollama pull llama3

# Download a specific size / variant using a tag after the colon
ollama pull llama3.2:1b
ollama pull gemma3:4b
```

The part after the colon is the **tag**. It usually indicates the parameter size (`1b` = 1 billion parameters, `8b`, `70b` ...) or a variant (`instruct`, `q4_0` quantization, and so on). If you omit the tag, Ollama pulls the default tag, which is normally a sensible mid-size version.

Term	Meaning in plain English
Parameters (e.g. 8b)	Roughly the "brain size" of the model. More parameters generally means smarter answers but more RAM/VRAM and slower speed.
Quantization (e.g. q4_0)	A compression technique that shrinks the model so it fits in less memory, trading a little accuracy for big size and speed savings.
Tag (after the <code>:</code>)	A specific version/size of a model, exactly like a Docker image tag.

DISK SPACE & DOWNLOAD TIME

Models are large — anywhere from a few hundred megabytes to tens of gigabytes. A good rule of thumb is to keep **at least twice the model's size** free on disk. A 4-billion parameter model is typically 2–3 GB; a 70-billion model can exceed 40 GB.

Pulling is also incremental: Ollama splits models into layers, so if two models share layers, the common parts are only downloaded once.

1.4 — Running a model in the terminal: `ollama run`

This is the command that makes everything feel real. `ollama run` starts an interactive chat session with a model right in your terminal. If the model isn't downloaded yet, `run` pulls it for you automatically first.

```
# Start chatting with Mistral (downloads it first if needed)
ollama run mistral
>>> Hello! Who are you?
I'm Mistral, a large language model running locally on your machine...
>>> /bye
```

Once inside the `>>>` prompt you can type questions naturally. A few useful in-session commands:

Type this at <code>>>></code>	What it does
<code>/bye</code>	Exit the chat session
<code>/?</code>	Show all available in-session commands
<code>/set parameter num_ctx 8192</code>	Change a setting (here, the context window) on the fly
<code>"""</code>	Begin multi-line input; close with <code>"""</code> on its own line

You can also run a model in **one shot** — pass the prompt directly and get a single answer without entering the interactive session. This is perfect for scripting:

```
# One-shot prompt, prints the answer and exits
ollama run gemma3 "Explain what an API is in one sentence."
```

MULTIMODAL IN ONE LINE

Vision-capable models can read an image straight from the command line by including a file path: `ollama run gemma3 "What's in this image? /path/to/photo.png"`. We cover multimodal models fully in Phase 6.

1.5 — Listing installed models: `ollama list`

To see everything you've downloaded, ask for the list. `ollama list` and its shorthand `ollama ls` are identical.

```
ollama list
NAME                ID                SIZE    MODIFIED
llama3:latest       365c0bd3c000     4.7 GB  2 days ago
gemma3:4b           a2af6cc3eb7f     3.3 GB  5 hours ago
mistral:latest      61e88e884507     4.1 GB  1 week ago
```


The columns tell you the model's name and tag, a unique content ID, how much disk it occupies, and when you last pulled or modified it. This is your inventory — handy before you decide what to keep or delete.

1.6 — Removing a model: `ollama rm`

Models take real disk space, so cleaning up matters. `ollama rm` permanently deletes a model's files from disk.

```
# Delete a model by its full name (include the tag if you used one)
ollama rm mistral
deleted 'mistral'

# You can remove several at once
ollama rm llama3:8b gemma3:1b
```

THIS IS IRREVERSIBLE

Removing a model deletes the downloaded weights. To use it again you'll need to `pull` it from scratch. Run `ollama list` first to confirm the exact name and tag you intend to delete.

1.7 — Checking running models: `ollama ps`

There's a difference between a model being *installed* (on disk) and a model being *loaded* (sitting in memory, ready to answer instantly). `ollama ps` shows what's currently loaded — much like the Unix `ps` command shows running processes.

```
ollama ps
NAME      ID          SIZE  PROCESSOR  UNTIL
llama3:70b bcfb190ca3a7 42 GB  100% GPU   4 minutes from now
```

The **PROCESSOR** column is especially useful — it tells you *where* the model is running:

PROCESSOR value	What it means
100% GPU	The whole model is loaded into your GPU's VRAM — fastest.
100% CPU	The whole model is in system RAM, running on the CPU — slower.
48%/52% CPU/GPU	The model was split across both because it didn't fully fit in VRAM.

The **UNTIL** column shows when the model will be unloaded. By default Ollama keeps a model in memory for **5 minutes** after its last use, so repeat requests stay fast. To unload immediately and free memory:

```
ollama stop llama3:70b
```

DID MY MODEL USE THE GPU?

`ollama ps` is the official way to confirm GPU usage. If you expected `100% GPU` but see CPU, the model was too large for your VRAM, or your GPU drivers aren't set up. We troubleshoot this in later phases.

1.8 — Where models are stored on disk

Knowing where the files live helps with backups, freeing space, or moving models to a larger drive. The default locations are:

Operating System	Default model directory
macOS	<code>~/.ollama/models</code>
Linux	<code>/usr/share/ollama/.ollama/models</code>
Windows	<code>C:\Users\<username>\.ollama\models</code>

Want them somewhere else — say a roomy external SSD? Set the `OLLAMA_MODELS` environment variable to a directory of your choice and restart Ollama.

```
# Example (Linux/macOS): store models on a different drive
export OLLAMA_MODELS=/mnt/bigdrive/ollama-models
```

LINUX PERMISSIONS NOTE

On Linux installed via the standard script, the `ollama` system user needs read and write access to that folder. Grant it with: `sudo chown -R ollama:ollama /mnt/bigdrive/ollama-models`.

1.9 — The different types of Ollama models

"Model" is a broad word. The Ollama library hosts several distinct *categories*, each built for a different job. Knowing them prevents the classic beginner mistake of asking a chat model to produce embeddings, or expecting a tiny model to write flawless code.

Category	What it's for	Examples on Ollama
General / Chat	Everyday conversation, writing, Q&A, summarising. Your default workhorse.	<code>llama3</code> , <code>gemma3</code> , <code>mistral</code> , <code>qwen</code>
Reasoning / "Thinking"	Models that work through problems step by step before answering — better at maths, logic and planning.	<code>deepseek-r1</code> , <code>gpt-oss</code>
Coding	Fine-tuned for writing, explaining and completing code.	<code>codellama</code> , <code>qwen3-coder</code>
Vision / Multimodal	Can read images alongside text — describe photos, read documents, answer visual questions.	<code>llava</code> , <code>gemma3</code> (vision variants)
Embedding	Turn text into numeric vectors for search and Retrieval-Augmented Generation (RAG). They do <i>not</i> chat.	<code>nomic-embed-text</code> , <code>embeddinggemma</code>
Tool-calling	Chat models trained to call external functions/tools, the basis of agents.	<code>llama3.1</code> , <code>qwen3</code>

Within each category, the same model usually comes in several **sizes** (e.g. 1B, 4B, 8B, 70B). Smaller = faster and lighter; larger = more capable but heavier. Start small while learning, then scale up once you know what your hardware can handle.

HOW TO CHOOSE YOUR FIRST MODEL

If you're unsure, start with `gemma3` or `llama3.2` in a small size (around 1–4B). They run comfortably on most laptops, respond quickly, and are more than capable for learning every technique in this guide.

Phase 1 command cheat sheet

Every command introduced in this phase, in one place for quick reference.

Command	Purpose
<code>ollama serve</code>	Start the background server (usually automatic)
<code>ollama --version</code>	Show the installed Ollama version
<code>ollama pull <model></code>	Download a model from the library
<code>ollama run <model></code>	Chat with a model interactively (pulls if missing)
<code>ollama run <model> "prompt"</code>	Get a single one-shot answer
<code>ollama list</code> / <code>ollama ls</code>	List all installed models
<code>ollama ps</code>	Show models currently loaded in memory
<code>ollama stop <model></code>	Unload a model from memory now
<code>ollama rm <model></code>	Delete a model from disk
<code>ollama show <model></code>	Display a model's details and parameters
<code>ollama cp <src> <dest></code>	Copy/duplicate a model under a new name

Build Your Local AI Lab — a model-management warm-up

Theory sticks when you use it. This short project walks you through every command above in a sensible order, ending with a tiny health-check script you can reuse. No Python yet — this is pure command line, so you cement the fundamentals before Phase 2.

Step 1 — Confirm the engine is alive

Check the version and ping the server. You should get a version string and "Ollama is running".

```
ollama --version
curl http://localhost:11434
```

Step 2 — Pull two small models to compare

Grab one general model and one reasoning model, both in small sizes so they download fast.

```
ollama pull gemma3:1b
ollama pull llama3.2:1b
```

Step 3 — Take inventory

```
ollama list
```

Step 4 — Run the same prompt through both, one-shot

Notice the difference in tone, speed and answer style — this is your first taste of how models differ.

```
ollama run gemma3:1b "Explain recursion to a 10-year-old in 3 sentences."
ollama run llama3.2:1b "Explain recursion to a 10-year-old in 3 sentences."
```

Step 5 — See what's loaded, then unload

```
ollama ps
ollama stop gemma3:1b
ollama stop llama3.2:1b
```

Step 6 — Save a reusable health-check script

Create a file called `ai-lab-status.sh` (macOS/Linux) and paste this in. Running it gives you an instant status report of your local lab.

```
#!/usr/bin/env bash
# ai-lab-status.sh – quick health check for your local Ollama lab

echo "=== Ollama version ==="
ollama --version

echo "=== Server heartbeat ==="
curl -s http://localhost:11434 || echo "Server not responding"

echo "=== Installed models ==="
ollama list

echo "=== Currently loaded in memory ==="
ollama ps
```

Make it executable and run it:

```
chmod +x ai-lab-status.sh
./ai-lab-status.sh
```

WINDOWS USERS

Save the equivalent commands in a `ai-lab-status.ps1` PowerShell file (the `ollama` commands are identical; replace `curl` with `curl.exe` or `Invoke-RestMethod http://localhost:11434`).

What you achieved

You installed Ollama, verified the local server on port 11434, downloaded and compared two models, inspected memory usage, freed it up, and built a reusable status script. You now own a working local AI lab — and in Phase 2 we'll start driving it from Python.

COMING NEXT — PHASE 2: PYTHON BASICS WITH OLLAMA

You'll install the `ollama` Python library, send your first programmatic prompts with `generate()` and `chat()`, learn the exact shape of the response object, and manage models entirely from Python.